HB-100 Pi 4 connection

11/30/2025

Hb-100 microwave sensors connection with the pi 4 for these projects

For connecting an HB-100 microwave sensor to a Raspberry Pi 4 for your human detecti…

viewing the output fore these

Since the HB-100 outputs an analog AC signal, you cannot directly “see” motion on the …

matplot to view on the dashboard

Perfect — we can integrate Matplotlib plots into your web dashboard so you can view th…

detcetion alert system python view

Got it — you want a Python-based detection alert system that reads your sensors (HB-1…

1

Perfect! Let’s build a complete all-in-one Python dashboard for your Raspberry Pi 4 that…

Combine with Matplotlib/Chart.js to plot HB-100 or thermal readings on dashboard

Perfect! Let’s combine Matplotlib and Chart.js so you can visualize HB-100 radar reading…

upgrade

Perfect! Let’s upgrade your dashboard into a fully integrated all-in-one interface for you…

yes

Perfect! Let’s upgrade your dashboard to overlay a thermal heatmap directly on the live…

threat level score

Perfect! Let’s implement a Threat Level Scoring System for your dashboard. This will…

Hb-100 microwave sensors connection with the pi 4 for these projects

For connecting an **HB-100 microwave sensor** to a Raspberry Pi 4 for your human detection project, you'll typically want to use it for motion detection and integrate it with your other sensors (thermal, PIR, camera). The HB-100 is a Doppler radar module that outputs a small AC signal corresponding to motion.

Here's a detailed connection and setup plan:

---

1. HB-100 Pinout

The HB-100 usually has three main pins:

1. **Vcc (5V)** – Power supply
2. **GND** – Ground
3. **IF / OUT** – Analog signal output (motion signal)

Some HB-100 boards also have a **LO (local oscillator)** and optional antenna connectors, but the main focus is Vcc, GND, and IF.

---

2. Powering HB-100

- Connect **Vcc → 5V** pin on Pi 4 (Pin 2 or Pin 4)
- Connect **GND → GND** on Pi 4 (Pin 6, 9, 14, etc.)
- Ensure your Pi power supply can handle additional current (~40-50 mA per HB-100).

---

3. Signal Connection

The IF output of HB-100 is an **analog signal** (~100–200 mV amplitude AC). Since Pi 4 **does not have ADC**, you'll need:

1. **Option A – ADC Module**
   Use an ADC like **MCP3008** to read the analog output.
   - IF → ADC channel input
   - MCP3008 communicates with Pi via SPI
   - Read the analog value in Python
2. **Option B – Signal Conditioning for GPIO**
   - Use a **comparator or op-amp** to convert the AC output into digital pulses

- Output can then go to a Pi GPIO pin for motion detection

---

4. Example Wiring (with MCP3008)

mathematica Copy code

```
HB-100 Vcc → Pi 5V
HB-100 GND → Pi GND
HB-100 IF  → MCP3008 CH0
MCP3008 VDD/VREF → Pi 3.3V
MCP3008 GND → Pi GND
MCP3008 CLK → Pi SCLK (GPIO11)
MCP3008 DOUT → Pi MISO (GPIO9)
MCP3008 DIN → Pi MOSI (GPIO10)
MCP3008 CS → Pi CE0 (GPIO8)
```

---

5. Python Code Snippet

python Copy code

```python
import spidev
import time

# Setup SPI
spi = spidev.SpiDev()
spi.open(0, 0)  # bus 0, device 0
spi.max_speed_hz = 1350000

def read_channel(channel):
    adc = spi.xfer2([1, (8+channel)<<4, 0])
    data = ((adc[1]&3) << 8) + adc[2]
    return data

while True:
    value = read_channel(0)
    if value > 100:  # threshold for motion
        print("Motion Detected!")
    time.sleep(0.1)
```

---

6. Integration Tips

- Fuse the HB-100 data with PIR, thermal, and camera detection for **robust human detection**.
- You can use a **threaded Python pipeline** where HB-100 updates a motion flag.

- Combine with YOLOv8 for **thermal + visual + radar fusion** as planned in your hackathon project 5_day_hackathon_plan.

If you want, I can draw a **complete wiring diagram of the HB-100 + Pi 4 + MCP3008 + other sensors** showing exactly how to integrate it into your 5-day hackathon setup. This would match your thermal, PIR, and camera connections.

Do you want me to do that?

viewing the output fore these

Since the HB-100 outputs an **analog AC signal**, you cannot directly "see" motion on the Pi without some processing. Depending on your setup, here's how you can view or monitor the output:

---

1. Using MCP3008 (ADC) + Python Console

- Read the analog signal via MCP3008 as shown in the previous code.
- You can **print raw values** in the console:

```python
value = read_channel(0)
print("HB-100 raw ADC:", value)
```

- When motion is detected, you will see **fluctuations or spikes** in the ADC value.

---

2. Graphing Real-Time Data

- Use **Matplotlib** to plot the signal in real-time:

```python
import matplotlib.pyplot as plt
import time

values = []

plt.ion()
fig, ax = plt.subplots()
line, = ax.plot(values)
```

```
while True:
    val = read_channel(0)
    values.append(val)
    if len(values) > 100:  # keep last 100 readings
        values.pop(0)
    line.set_ydata(values)
    line.set_xdata(range(len(values)))
    ax.relim()
    ax.autoscale_view()
    plt.pause(0.01)
```

- You'll see spikes whenever the HB-100 detects motion.

---

3. Digital Pulse via Comparator

- If you convert the AC signal to a **digital high/low pulse**, you can:
  - Connect it to a GPIO pin
  - Use `` `GPIO.input()` `` to detect motion
  - Display a **text alert** on console: `` `Motion Detected!` ``
  - Trigger **LED, buzzer, or dashboard alert** in real-time.

---

4. Integration with Dashboard

- For your hackathon project, you can integrate the HB-100 output into your **Flask + WebRTC dashboard**:
  - Motion → update a **motion flag** in Python
  - Use WebSocket to send motion events to browser
  - Show an **overlay indicator** when motion is detected alongside camera and thermal feed
    5_day_hackathon_plan
    .

💡 **Tip:** To visualize the HB-100 signal like a "heatmap" of motion:

- Combine its detection with your **thermal sensor output**
- Overlay motion detection as **blinking or highlighted areas** on camera feed.

If you want, I can create a **ready-to-use Python dashboard code** for Pi 4 that **shows live camera feed + thermal heatmap + HB-100 motion spikes** all in one window.

Do you want me to do that?

matplot to view on the dashboard

Perfect — we can integrate **Matplotlib plots into your web dashboard** so you can view the HB-100 sensor output in real-time alongside your camera and thermal feeds. Since Flask doesn't natively embed Matplotlib dynamically, the typical approach is to **update the plot as an image and refresh it** or use **WebSockets for live data streaming**. Here's a step-by-step plan:

---

1. Python: Matplotlib Live Plot as Image

You can create a Matplotlib figure and save it to a PNG, which Flask serves to the browser.

python Copy code

```python
import matplotlib.pyplot as plt
import io
import base64
from flask import Flask, render_template
import random  # Replace with HB-100 read_channel()

app = Flask(__name__)
values = []

@app.route('/')
def dashboard():
    # Simulate reading HB-100
    value = random.randint(0, 1023)  # replace with read_channel(0)
    values.append(value)
    if len(values) > 100:
        values.pop(0)

    # Create plot
    fig, ax = plt.subplots()
    ax.plot(values)
    ax.set_ylim(0, 1023)
    ax.set_title("HB-100 Motion Sensor Output")

    # Save plot to PNG in-memory
    buf = io.BytesIO()
    fig.savefig(buf, format='png')
    buf.seek(0)
    plot_data = base64.b64encode(buf.getvalue()).decode()
    buf.close()

    # Render HTML with plot
    return f'<img src="{plot_data}"/>'
```

```
if __name__ == "__main__":
    app.run(host='0.0.0.0', port=5000, debug=True)
```

✅ This will show a **live-updating plot** every time you refresh the page.

---

2. Real-Time Updates with WebSockets

For a more dynamic dashboard (like your thermal + camera feeds):

1. Use **Flask-SocketIO** to push HB-100 readings to the browser.
2. Use **Chart.js or Plotly.js** on the frontend to render live plots.

Backend (Python):

python Copy code

```
from flask_socketio import SocketIO, emit
import threading, time

socketio = SocketIO(app)

def sensor_thread():
    while True:
        value = read_channel(0)  # HB-100 reading
        socketio.emit('hb100_data', {'value': value})
        time.sleep(0.1)

threading.Thread(target=sensor_thread).start()
```

Frontend (HTML + JS):

html Copy code

```
<canvas id="hb100Plot" width="400" height="200"></canvas>
<script src="//cdn.jsdelivr.net/npm/chart.js"></script>
<script src="//cdnjs.cloudflare.com/ajax/libs/socket.io/4.7.1/socket.io.min.js"></script>
<script>
var ctx = document.getElementById('hb100Plot').getContext('2d');
var data = { labels: [], datasets: [{ label: 'HB-100', data: [], borderColor: 'red', fill:
false }] };
var chart = new Chart(ctx, { type: 'line', data: data, options: { animation: false } });

var socket = io();
socket.on('hb100_data', function(msg){
    data.labels.push('');
    data.datasets[0].data.push(msg.value);
    if(data.labels.length > 100){
        data.labels.shift();
        data.datasets[0].data.shift();
```

```
    }
    chart.update();
});
</script>
```

✅ This gives a **true live plot** alongside your other feeds without refreshing the page.

I can create a **full Flask + WebRTC + Thermal + HB-100 dashboard template** with live Matplotlib/Chart.js plotting integrated for your hackathon, so all sensors show in **one window**.

Do you want me to do that next?

detcetion alert system python view

Got it — you want a **Python-based detection alert system** that reads your sensors (HB-100 radar, PIR, thermal, and camera with YOLO), processes them, and **triggers alerts** (console, buzzer, LED, or web dashboard). Here's a **ready-to-use architecture and Python implementation**.

---

1. System Overview

Sensors:

- HB-100 → Motion (radar)
- PIR → Presence
- Thermal sensor → Heat signatures
- Pi Camera → Visual feed for YOLO detection

Alert Outputs:

- Console log
- Buzzer or LED via GPIO
- Dashboard / Web alert via Flask + SocketIO / MQTT

Logic:

- Fuse sensors for higher accuracy:
  - `HB-100 OR PIR OR YOLO detection OR thermal threshold` → **Alert**
- Optional "confidence score" based on multiple sensors triggering simultaneously

---

2. Python Alert System Example

python

Copy code

```
import RPi.GPIO as GPIO
import time
import threading
from random import randint  # Replace with real sensor readings
import cv2  # For camera + YOLO (mocked here)
from flask import Flask
from flask_socketio import SocketIO

# ---------------- GPIO Setup ----------------
BUZZER_PIN = 18
GPIO.setmode(GPIO.BCM)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

# ---------------- Flask / Dashboard ----------------
app = Flask(__name__)
socketio = SocketIO(app)

# ---------------- Sensor Reading Functions ----------------
def read_hb100():
    # Replace with MCP3008 read_channel(0)
    return randint(0, 1023)

def read_pir():
    # Replace with GPIO.input(PIR_PIN)
    return randint(0, 1)

def read_thermal():
    # Replace with actual thermal sensor reading
    return randint(20, 50)

def read_camera_yolo():
    # Replace with actual YOLO detection
    return randint(0, 1)  # 1 = human detected

# ---------------- Alert Function ----------------
def alert():
    print("ALERT! Human Detected!")
    GPIO.output(BUZZER_PIN, GPIO.HIGH)
    time.sleep(0.5)
    GPIO.output(BUZZER_PIN, GPIO.LOW)
    socketio.emit('alert', {'message': 'Human Detected!'})

# ---------------- Sensor Monitoring Thread ----------------
def monitor_sensors():
    while True:
        hb_val = read_hb100()
        pir_val = read_pir()
        thermal_val = read_thermal()
```

```
        camera_val = read_camera_yolo()

        # Simple fusion logic
        if hb_val > 500 or pir_val == 1 or thermal_val > 35 or camera_val == 1:
            alert()

        time.sleep(0.1)  # 10 Hz monitoring

# ---------------- Flask Routes ----------------
@app.route('/')
def index():
    return "<h1>Human Detection Dashboard</h1>"

# ---------------- Run ----------------
threading.Thread(target=monitor_sensors, daemon=True).start()
socketio.run(app, host='0.0.0.0', port=5000)
```

---

3. Features of This System

- **Real-time monitoring** of HB-100, PIR, thermal, and camera sensors
- **Alerts fused from multiple sensors**
- **Console + buzzer + dashboard alerts**
- **Extendable**: Add YOLO bounding boxes or thermal heatmap overlay
- **Runs on Raspberry Pi 4** with minimal latency

💡 **Tips for Hackathon:**

1. Combine with **Matplotlib/Chart.js** to plot HB-100 or thermal readings on dashboard.
2. Use **YOLOv8 Nano** for fast human detection on Pi.
3. Implement **threat-level scoring** based on how many sensors triggered.

I can create a **complete “all-in-one” Python dashboard** for your hackathon that shows:

- Live camera feed with YOLO boxes
- Thermal heatmap
- HB-100 + PIR sensor live plot
- Alerts (LED, buzzer, web notification)

...all running together on the Pi 4.

Do you want me to do that next?

1

Perfect! Let's build a **complete all-in-one Python dashboard for your Raspberry Pi 4** that integrates:

- **Live camera feed with YOLO bounding boxes**
- **Thermal heatmap overlay**
- **HB-100 + PIR sensor live plots**
- **Real-time alerts** (buzzer, LED, web notifications)

Below is a **full Python + Flask + SocketIO dashboard template**.

---

1. Project Structure

```php
hackathon_dashboard/
│
├── app.py                # Main Flask + SocketIO server
├── templates/
│   └── index.html        # Dashboard HTML
├── static/
│   └── scripts.js        # Frontend JS for plots & alerts
└── sensors.py            # Sensor reading functions (HB-100, PIR, Thermal, Camera)
```

---

2. sensors.py – Read sensors

```python
import random
import cv2

# ---- Camera + YOLO setup ----
def init_camera():
    cam = cv2.VideoCapture(0)
    return cam

def read_camera_yolo(cam):
    ret, frame = cam.read()
    # Replace this with actual YOLO inference
    human_detected = random.randint(0, 1)
    return frame, human_detected

# ---- Thermal sensor ----
```

```python
def read_thermal():
    # Replace with actual sensor reading
    return random.randint(20, 50)

# ---- HB-100 ----
def read_hb100():
    # Replace with MCP3008 read_channel(0)
    return random.randint(0, 1023)

# ---- PIR sensor ----
def read_pir():
    # Replace with GPIO.input(PIR_PIN)
    return random.randint(0, 1)
```

---

3. app.py – Flask + SocketIO Server

python Copy code

```python
from flask import Flask, render_template
from flask_socketio import SocketIO
import threading, time
import RPi.GPIO as GPIO
from sensors import init_camera, read_camera_yolo, read_thermal, read_hb100, read_pir

# ---- GPIO Setup ----
BUZZER_PIN = 18
GPIO.setmode(GPIO.BCM)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

# ---- Flask App ----
app = Flask(__name__)
socketio = SocketIO(app)

# ---- Initialize Camera ----
camera = init_camera()

# ---- Alert Function ----
def alert():
    GPIO.output(BUZZER_PIN, GPIO.HIGH)
    time.sleep(0.3)
    GPIO.output(BUZZER_PIN, GPIO.LOW)
    socketio.emit('alert', {'message': 'Human Detected!'})

# ---- Sensor Monitoring Thread ----
def monitor_sensors():
    while True:
        hb_val = read_hb100()
```

```
        pir_val = read_pir()
        thermal_val = read_thermal()
        _, cam_detected = read_camera_yolo(camera)

        # Fusion logic
        if hb_val > 500 or pir_val == 1 or thermal_val > 35 or cam_detected == 1:
            alert()

        # Emit live sensor data to frontend
        socketio.emit('sensor_data', {
            'hb100': hb_val,
            'pir': pir_val,
            'thermal': thermal_val,
            'camera': cam_detected
        })

        time.sleep(0.1)

# ---- Routes ----
@app.route('/')
def index():
    return render_template('index.html')

# ---- Run ----
threading.Thread(target=monitor_sensors, daemon=True).start()
socketio.run(app, host='0.0.0.0', port=5000)
```

4. templates/index.html – Dashboard

html Copy code

```
<!DOCTYPE html>
<html>
<head>
    <title>Human Detection Dashboard</title>
    <script src="//cdnjs.cloudflare.com/ajax/libs/socket.io/4.7.1/socket.io.min.js"></script>
    <script src="//cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
<h1>Human Detection Dashboard</h1>
<canvas id="sensorPlot" width="600" height="300"></canvas>
<div id="alerts"></div>

<script>
var socket = io();

var ctx = document.getElementById('sensorPlot').getContext('2d');
```

```
var data = {
    labels: [],
    datasets: [
        {label: 'HB-100', data: [], borderColor: 'red', fill: false},
        {label: 'Thermal', data: [], borderColor: 'orange', fill: false},
        {label: 'PIR', data: [], borderColor: 'green', fill: false},
        {label: 'Camera', data: [], borderColor: 'blue', fill: false},
    ]
};
var chart = new Chart(ctx, { type: 'line', data: data, options: {animation:false} });

socket.on('sensor_data', function(msg){
    data.labels.push('');
    data.datasets[0].data.push(msg.hb100);
    data.datasets[1].data.push(msg.thermal);
    data.datasets[2].data.push(msg.pir);
    data.datasets[3].data.push(msg.camera);

    if(data.labels.length > 100){
        data.labels.shift();
        data.datasets.forEach(ds => ds.data.shift());
    }
    chart.update();
});

socket.on('alert', function(msg){
    document.getElementById('alerts').innerHTML = "<h2 style='color:red'>"+msg.message+"
</h2>";
});
</script>
</body>
</html>
```

---

5. Features

- Live plotting of **HB-100, PIR, Thermal, Camera detection**
- **Real-time alerts** (buzzer + dashboard text)
- Easily extendable to **thermal heatmaps + YOLO bounding boxes**
- Runs **entirely on Pi 4** with low latency

If you want, I can **upgrade this dashboard** to also **overlay thermal heatmaps and camera YOLO bounding boxes** in **one live video feed window** — making it exactly like your 5-day hackathon prototype 5_day_hackathon_plan.

Do you want me to do that next?

Combine with Matplotlib/Chart.js to plot HB-100 or thermal readings on dashboard

Perfect! Let's combine **Matplotlib** and **Chart.js** so you can visualize **HB-100 radar readings, thermal sensor values, and other sensor data** on your dashboard in real time.

We'll use this approach:

- **Backend (Python + Flask + SocketIO)**: Reads sensors and emits data to the frontend.
- **Frontend (Chart.js)**: Displays live plots for HB-100, thermal, PIR, and camera detection.
- **Optional Matplotlib**: Can generate static images for historical or snapshot views.

---

1. Backend: app.py

python Copy code

```python
from flask import Flask, render_template
from flask_socketio import SocketIO
import threading, time
import random  # Replace with actual sensor functions
import RPi.GPIO as GPIO

# GPIO for buzzer/LED
BUZZER_PIN = 18
GPIO.setmode(GPIO.BCM)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

# Flask + SocketIO
app = Flask(__name__)
socketio = SocketIO(app)

# Sensor simulation (replace with real reads)
def read_hb100(): return random.randint(0,1023)
def read_thermal(): return random.randint(20,50)
def read_pir(): return random.randint(0,1)
def read_camera(): return random.randint(0,1)

# Alert function
def alert():
    GPIO.output(BUZZER_PIN, GPIO.HIGH)
    time.sleep(0.2)
```

```
    GPIO.output(BUZZER_PIN, GPIO.LOW)
    socketio.emit('alert', {'message':'Human Detected!'})

# Sensor monitoring thread
def monitor_sensors():
    while True:
        hb_val = read_hb100()
        thermal_val = read_thermal()
        pir_val = read_pir()
        cam_val = read_camera()

        # Fusion logic for alert
        if hb_val>500 or thermal_val>35 or pir_val==1 or cam_val==1:
            alert()

        # Emit live data to frontend
        socketio.emit('sensor_data', {
            'hb100': hb_val,
            'thermal': thermal_val,
            'pir': pir_val,
            'camera': cam_val
        })
        time.sleep(0.1)

@app.route('/')
def index(): return render_template('index.html')

# Start monitoring thread
threading.Thread(target=monitor_sensors, daemon=True).start()
socketio.run(app, host='0.0.0.0', port=5000)
```

2. Frontend: templates/index.html

html Copy code

```
<!DOCTYPE html>
<html>
<head>
    <title>Human Detection Dashboard</title>
    <script src="//cdnjs.cloudflare.com/ajax/libs/socket.io/4.7.1/socket.io.min.js"></script>
    <script src="//cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
<h1>Human Detection Dashboard</h1>
<canvas id="sensorPlot" width="800" height="300"></canvas>
<div id="alerts"></div>
```

```
<script>
var ctx = document.getElementById('sensorPlot').getContext('2d');
var data = {
    labels: [],
    datasets: [
        {label:'HB-100', data:[], borderColor:'red', fill:false},
        {label:'Thermal', data:[], borderColor:'orange', fill:false},
        {label:'PIR', data:[], borderColor:'green', fill:false},
        {label:'Camera', data:[], borderColor:'blue', fill:false}
    ]
};
var chart = new Chart(ctx, {type:'line', data:data, options:{animation:false}});

var socket = io();
socket.on('sensor_data', function(msg){
    data.labels.push('');
    data.datasets[0].data.push(msg.hb100);
    data.datasets[1].data.push(msg.thermal);
    data.datasets[2].data.push(msg.pir);
    data.datasets[3].data.push(msg.camera);

    if(data.labels.length>100){
        data.labels.shift();
        data.datasets.forEach(ds => ds.data.shift());
    }
    chart.update();
});

socket.on('alert', function(msg){
    document.getElementById('alerts').innerHTML = "<h2 style='color:red'>"+msg.message+"
</h2>";
});
</script>
</body>
</html>
```

---

3. Optional: Matplotlib Snapshot Images

If you want to capture **Matplotlib snapshots of the HB-100/thermal readings**:

python Copy code

```
import matplotlib.pyplot as plt
import io, base64

def generate_snapshot(hb_values, thermal_values):
    plt.figure()
    plt.plot(hb_values, label='HB-100', color='red')
```

```
    plt.plot(thermal_values, label='Thermal', color='orange')
    plt.legend()
    buf = io.BytesIO()
    plt.savefig(buf, format='png')
    buf.seek(0)
    img_b64 = base64.b64encode(buf.getvalue()).decode()
    plt.close()
    return img_b64
```

- Then you can embed `img_b64` in HTML like:

```html
<img src="{{ snapshot }}">
```

✅ Features of this Dashboard

- Live **Chart.js plotting** of HB-100, thermal, PIR, camera
- **Real-time alerts** on dashboard + buzzer
- Optional **Matplotlib snapshots** for logging or historical views
- Can be combined later with **YOLO camera feed + thermal heatmap overlay**

If you want, I can **upgrade this setup** to show **live camera feed with YOLO boxes + thermal heatmap overlay + HB-100/thermal Chart.js plots** all in a **single dashboard window**, which will be exactly like your hackathon prototype.

Do you want me to do that next?

upgrade

Perfect! Let's upgrade your dashboard into a **fully integrated all-in-one interface** for your hackathon prototype. The upgraded dashboard will include:

1. **Live Camera Feed with YOLO human detection bounding boxes**
2. **Thermal Heatmap overlay**
3. **Live HB-100 + PIR + Thermal sensor plots using Chart.js**
4. **Real-time alerts** (buzzer + dashboard notification)

This way, all sensor data and detection results are **visualized in one window** on your Raspberry Pi 4.

---

1. Project Structure

```php
hackathon_dashboard/
│
├── app.py               # Flask + SocketIO server
├── sensors.py           # Sensor reading functions (camera, thermal, HB-100, PIR)
├── templates/
│   └── index.html       # Dashboard HTML
├── static/
│   └── scripts.js       # Frontend JS for plots & alerts
├── yolo_model/          # Your YOLOv8 / YOLOv5 model weights
└── thermal_data/        # Optional thermal readings log
```

---

2. sensors.py

```python
import cv2
import random  # Replace with real sensor reads

# Camera + YOLO
def init_camera():
    cam = cv2.VideoCapture(0)
    return cam

def read_camera_yolo(cam):
    ret, frame = cam.read()
    # Placeholder for YOLO detection
    # Replace with actual YOLO inference
    human_detected = random.randint(0,1)
    # For demo: draw random bounding box
    if human_detected:
        h, w, _ = frame.shape
        cv2.rectangle(frame, (w//4,h//4), (w//2,h//2), (0,0,255), 2)
    return frame, human_detected

# Thermal sensor
def read_thermal():
    return random.randint(20,50)

# HB-100 radar
def read_hb100():
    return random.randint(0,1023)

# PIR sensor
```

```python
def read_pir():
    return random.randint(0,1)
```

---

3. app.py

python Copy code

```python
from flask import Flask, render_template, Response
from flask_socketio import SocketIO
import threading, time, cv2, base64
import RPi.GPIO as GPIO
from sensors import init_camera, read_camera_yolo, read_thermal, read_hb100, read_pir

# GPIO setup
BUZZER_PIN = 18
GPIO.setmode(GPIO.BCM)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

# Flask + SocketIO
app = Flask(__name__)
socketio = SocketIO(app)

camera = init_camera()

# Alert
def alert():
    GPIO.output(BUZZER_PIN, GPIO.HIGH)
    time.sleep(0.2)
    GPIO.output(BUZZER_PIN, GPIO.LOW)
    socketio.emit('alert', {'message':'Human Detected!'})

# Sensor monitoring thread
def monitor_sensors():
    while True:
        hb_val = read_hb100()
        pir_val = read_pir()
        thermal_val = read_thermal()
        _, cam_detected = read_camera_yolo(camera)

        # Fusion logic
        if hb_val>500 or pir_val==1 or thermal_val>35 or cam_detected==1:
            alert()

        # Emit sensor data to frontend
        socketio.emit('sensor_data', {
            'hb100': hb_val,
            'pir': pir_val,
```

```
            'thermal': thermal_val,
            'camera': cam_detected
        })
        time.sleep(0.1)

# Camera streaming generator
def gen_frames():
    while True:
        frame, _ = read_camera_yolo(camera)
        _, buffer = cv2.imencode('.jpg', frame)
        frame_bytes = buffer.tobytes()
        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + frame_bytes + b'\r\n')

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/video_feed')
def video_feed():
    return Response(gen_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')

# Start sensor monitoring thread
threading.Thread(target=monitor_sensors, daemon=True).start()
socketio.run(app, host='0.0.0.0', port=5000)
```

4. templates/index.html

html Copy code

```
<!DOCTYPE html>
<html>
<head>
    <title>Human Detection Dashboard</title>
    <script src="//cdnjs.cloudflare.com/ajax/libs/socket.io/4.7.1/socket.io.min.js"></script>
    <script src="//cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
<h1>Human Detection Dashboard</h1>

<!-- Live Camera + Thermal Heatmap -->
<img src="{{ url_for('video_feed') }}" width="640" height="480" />

<!-- Sensor Plots -->
<canvas id="sensorPlot" width="800" height="300"></canvas>
<div id="alerts"></div>
```

```
<script>
var ctx = document.getElementById('sensorPlot').getContext('2d');
var data = {
    labels: [],
    datasets: [
        {label:'HB-100', data:[], borderColor:'red', fill:false},
        {label:'Thermal', data:[], borderColor:'orange', fill:false},
        {label:'PIR', data:[], borderColor:'green', fill:false},
        {label:'Camera', data:[], borderColor:'blue', fill:false}
    ]
};
var chart = new Chart(ctx, {type:'line', data:data, options:{animation:false}});

var socket = io();
socket.on('sensor_data', function(msg){
    data.labels.push('');
    data.datasets[0].data.push(msg.hb100);
    data.datasets[1].data.push(msg.thermal);
    data.datasets[2].data.push(msg.pir);
    data.datasets[3].data.push(msg.camera);
    if(data.labels.length>100){
        data.labels.shift();
        data.datasets.forEach(ds => ds.data.shift());
    }
    chart.update();
});

socket.on('alert', function(msg){
    document.getElementById('alerts').innerHTML = "<h2 style='color:red'>"+msg.message+"
</h2>";
});
</script>
</body>
</html>
```

✅ Features of This Upgraded Dashboard

1. **Live video feed** from Pi Camera with YOLO detection bounding boxes
2. **Thermal sensor values overlayed on plots** (heatmap overlay can be added later)
3. **HB-100 radar & PIR sensor live Chart.js plotting**
4. **Real-time alerts**: buzzer + dashboard notifications
5. **All-in-one interface** for hackathon demo, showing multi-sensor fusion

---

I can further **upgrade this dashboard to overlay a thermal heatmap directly on the live camera feed** so you can **see humans as hot spots with YOLO boxes simultaneously**, which looks very polished for a hackathon presentation.

Do you want me to do that next?

yes

Perfect! Let's upgrade your dashboard to **overlay a thermal heatmap directly on the live camera feed with YOLO bounding boxes**, so you can **see humans as hot spots with detection boxes simultaneously**. This is exactly the kind of visual demo that wins hackathons.

We'll combine:

- **Pi Camera live feed**
- **YOLO detection boxes**
- **Thermal sensor heatmap overlay**
- **HB-100 & PIR Chart.js plots**
- **Real-time alerts**

---

1. sensors.py – Thermal Mapping

We will generate a small **thermal matrix** (from your thermal sensor or array) and map it onto the camera feed.

python Copy code

```python
import cv2
import numpy as np
import random

# Camera + YOLO
def init_camera():
    cam = cv2.VideoCapture(0)
    return cam

def read_camera_yolo(cam):
    ret, frame = cam.read()
    # Placeholder YOLO detection
    human_detected = random.randint(0, 1)
    if human_detected:
        h, w, _ = frame.shape
        cv2.rectangle(frame, (w//4,h//4), (w//2,h//2), (0,0,255), 2)
```

```
    return frame, human_detected

# Thermal sensor array simulation (replace with real sensor data)
def read_thermal_matrix():
    # Simulate 8x8 thermal sensor
    thermal_matrix = np.random.randint(20, 50, (8, 8))
    return thermal_matrix

# HB-100 radar
def read_hb100():
    return random.randint(0, 1023)

# PIR sensor
def read_pir():
    return random.randint(0,1)
```

2. app.py – Overlay Heatmap on Video

python Copy code

```
from flask import Flask, render_template, Response
from flask_socketio import SocketIO
import threading, time, cv2, numpy as np, RPi.GPIO as GPIO
from sensors import init_camera, read_camera_yolo, read_thermal_matrix, read_hb100, read_pir

# GPIO setup
BUZZER_PIN = 18
GPIO.setmode(GPIO.BCM)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

# Flask + SocketIO
app = Flask(__name__)
socketio = SocketIO(app)

camera = init_camera()

def alert():
    GPIO.output(BUZZER_PIN, GPIO.HIGH)
    time.sleep(0.2)
    GPIO.output(BUZZER_PIN, GPIO.LOW)
    socketio.emit('alert', {'message':'Human Detected!'})

# Sensor monitoring
def monitor_sensors():
    while True:
        hb_val = read_hb100()
        pir_val = read_pir()
```

```
        thermal_matrix = read_thermal_matrix()
        _, cam_detected = read_camera_yolo(camera)

        if hb_val>500 or pir_val==1 or np.max(thermal_matrix)>35 or cam_detected==1:
            alert()

        socketio.emit('sensor_data', {
            'hb100': hb_val,
            'pir': pir_val,
            'thermal': int(np.mean(thermal_matrix)),
            'camera': cam_detected
        })
        time.sleep(0.1)

# Video feed with thermal overlay
def gen_frames():
    while True:
        frame, _ = read_camera_yolo(camera)
        thermal_matrix = read_thermal_matrix()
        # Resize thermal to frame size
        heatmap = cv2.resize(thermal_matrix, (frame.shape[1], frame.shape[0]))
        heatmap_color = cv2.applyColorMap(np.uint8(heatmap*5), cv2.COLORMAP_JET)
        overlayed = cv2.addWeighted(frame, 0.7, heatmap_color, 0.3, 0)
        _, buffer = cv2.imencode('.jpg', overlayed)
        frame_bytes = buffer.tobytes()
        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + frame_bytes + b'\r\n')

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/video_feed')
def video_feed():
    return Response(gen_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')

threading.Thread(target=monitor_sensors, daemon=True).start()
socketio.run(app, host='0.0.0.0', port=5000)
```

3. templates/index.html – Full Dashboard

html Copy code

```
<!DOCTYPE html>
<html>
<head>
    <title>Human Detection Dashboard</title>
```

```
    <script src="//cdnjs.cloudflare.com/ajax/libs/socket.io/4.7.1/socket.io.min.js"></script>
    <script src="//cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
<h1>Human Detection Dashboard</h1>

<!-- Live Video Feed with Thermal Overlay -->
<img src="{{ url_for('video_feed') }}" width="640" height="480" />

<!-- Sensor Plots -->
<canvas id="sensorPlot" width="800" height="300"></canvas>
<div id="alerts"></div>

<script>
var ctx = document.getElementById('sensorPlot').getContext('2d');
var data = {
    labels: [],
    datasets: [
        {label:'HB-100', data:[], borderColor:'red', fill:false},
        {label:'Thermal', data:[], borderColor:'orange', fill:false},
        {label:'PIR', data:[], borderColor:'green', fill:false},
        {label:'Camera', data:[], borderColor:'blue', fill:false}
    ]
};
var chart = new Chart(ctx, {type:'line', data:data, options:{animation:false}});

var socket = io();
socket.on('sensor_data', function(msg){
    data.labels.push('');
    data.datasets[0].data.push(msg.hb100);
    data.datasets[1].data.push(msg.thermal);
    data.datasets[2].data.push(msg.pir);
    data.datasets[3].data.push(msg.camera);

    if(data.labels.length>100){
        data.labels.shift();
        data.datasets.forEach(ds => ds.data.shift());
    }
    chart.update();
});

socket.on('alert', function(msg){
    document.getElementById('alerts').innerHTML = "<h2 style='color:red'>"+msg.message+"
</h2>";
});
</script>
</body>
</html>
```

✅ Features of This Full Upgrade

1. **Live camera feed with YOLO human detection boxes**
2. **Thermal heatmap overlaid on camera feed**
3. **Live sensor plots** (HB-100, PIR, thermal, camera) using Chart.js
4. **Real-time alerts** on dashboard + buzzer
5. **All-in-one integrated interface** for hackathon demo

If you want, I can also **add a threat-level scoring system** based on the number of sensors triggered and **display it visually** on the dashboard — perfect for impressing judges.

Do you want me to do that next?

threat level score

Perfect! Let's implement a **Threat Level Scoring System** for your dashboard. This will assign a **numerical or color-coded threat level** based on how many sensors are triggered at once. This makes your hackathon demo look professional and polished.

---

1. Threat Level Logic

We can define a **score from 0 to 100** based on weighted sensor inputs:

Sensor	Condition	Weight
HB-100 radar	Value > threshold (e.g., 500)	25
PIR	Motion detected	25
Thermal sensor	Max temp > threshold (e.g., 35)	25
Camera YOLO	Human detected	25

Threat Level = sum of weights of triggered sensors

- **0** → No threat
- **25–50** → Low threat
- **51–75** → Medium threat
- **76–100** → High threat

We can also display **color-coded bar** on the dashboard.

---

2. Backend Changes – app.py

Add threat level calculation in the sensor monitoring thread:

python Copy code

```python
def calculate_threat(hb_val, pir_val, thermal_val, cam_detected):
    score = 0
    if hb_val > 500: score += 25
    if pir_val == 1: score += 25
    if thermal_val > 35: score += 25
    if cam_detected == 1: score += 25
    return score

def monitor_sensors():
    while True:
        hb_val = read_hb100()
        pir_val = read_pir()
        thermal_matrix = read_thermal_matrix()
        thermal_val = int(np.max(thermal_matrix))
        _, cam_detected = read_camera_yolo(camera)

        # Threat level
        threat_level = calculate_threat(hb_val, pir_val, thermal_val, cam_detected)

        # Alert if threat detected
        if threat_level > 0:
            alert()

        # Emit sensor data + threat level
        socketio.emit('sensor_data', {
            'hb100': hb_val,
            'pir': pir_val,
            'thermal': thermal_val,
            'camera': cam_detected,
            'threat': threat_level
        })
        time.sleep(0.1)
```

---

3. Frontend – index.html

Add a **threat level bar** under the chart:

html Copy code

```html
<div style="width:800px; height:30px; background:#ddd; margin-top:10px;">
    <div id="threatBar" style="height:100%; width:0%; background:red;"></div>
</div>
<p id="threatText"></p>
```

```
<script>
socket.on('sensor_data', function(msg){
    // Existing Chart.js updates
    // ...

    // Update threat bar
    var threatBar = document.getElementById('threatBar');
    var threatText = document.getElementById('threatText');
    threatBar.style.width = msg.threat + '%';
    if(msg.threat <= 25) threatBar.style.background = 'green';
    else if(msg.threat <= 50) threatBar.style.background = 'yellow';
    else if(msg.threat <= 75) threatBar.style.background = 'orange';
    else threatBar.style.background = 'red';

    var levelText = '';
    if(msg.threat <= 25) levelText = 'Low Threat';
    else if(msg.threat <= 50) levelText = 'Moderate Threat';
    else if(msg.threat <= 75) levelText = 'High Threat';
    else levelText = 'Critical Threat';

    threatText.innerHTML = 'Threat Level: ' + levelText + ' (' + msg.threat + '%)';
});
</script>
```

✅ Features Added

1. **Threat level score (0–100)** based on sensor fusion
2. **Color-coded bar** (green → red) showing threat visually
3. **Text description**: Low, Moderate, High, Critical
4. Integrates seamlessly with **existing camera + thermal + plots + alerts**

This gives judges a **clear numeric and visual representation** of threat at a glance, making your hackathon demo more professional.

If you want, I can **also overlay the threat level directly on the live camera feed with heatmap + YOLO boxes**, so it shows **one complete visual summary** in real-time.

Do you want me to do that next?